



FOR JAVA DEVELOPERS MOVING TO NESTJS

Production mode. Still exposed.

Spring Boot to NestJS, episode 32: GraphQL

Your GraphQL error contract depends on more than your resolver. Production mode hid the stack, not the database error.

A resolver whose SQL fails: Spring for GraphQL masked it as an internal error with an id for the logs.

NestJS with Apollo returned the raw Postgres message and a stack trace. With `NODE_ENV=production` the stack went and the message stayed.

A `formatError` that masks anything GraphQL itself did not raise made it "Internal server error".

In scope

- Schema first and code first
- N plus one and DataLoader
- Unexpected errors
- Status codes and introspection

Not in scope

- Everything in the Spring GraphQL flagship
- Federation

At the time of recording Nest uses Apollo Server 5 through [@nestjs/apollo](#).

01 The measurements

N PLUS ONE, BOTH STACKS	21 statements
@BatchMapping / DataLoader PER REQUEST	2 statements
SQL ERROR, SPRING	INTERNAL_ERROR with an id
SQL ERROR, NEST, NODE_ENV UNSET	Postgres message + 7 line stack
SQL ERROR, NEST, PRODUCTION	Postgres message, no stack
SQL ERROR, NEST, formatError MASK	Internal server error
UNKNOWN FIELD, ACCEPT JSON	Spring 200, Apollo 400

20 orders, 10 customers, one Postgres.

02 Make the error policy explicit

Mask any error GraphQL itself did not raise, log the original on the server, and send the client a generic message.

app.module.ts

```
GraphQLModule.forRoot<ApolloDriverConfig>({
  driver: ApolloDriver,
  autoSchemaFile: true,
  formatError: (formatted, error) => {
    const original = error instanceof GraphQLError ? error.originalError : undefined;
    if (!original || original instanceof GraphQLError) return formatted;
    logger.error(original);
    return { message: 'Internal server error', path: formatted.path,
      extensions: { code: 'INTERNAL_SERVER_ERROR' } };
  },
});
```

with the mask, NODE_ENV=production: "Internal server error", no stack trace

Run on a real machine, 2026-10-01, by scripts/verify-graphql.sh in the course repository.

03 A DataLoader per request

Nest has no @BatchMapping. Create the loader in the GraphQL context so each request batches its own lookups.

orders.resolver.ts

```
@ResolveField(() => Customer)
customer(@Parent() order: Order, @Context() ctx: { loaders: Loaders }) {
  return ctx.loaders.customers.load(order.customerId);
}
```

20 orders with their customer: 21 statements, then 2 with the loader

Run on a real machine, 2026-10-01, by scripts/verify-graphql.sh in the course repository.

Before you move a GraphQL API to Nest

1. What does the client see when a resolver throws something unexpected?
2. Is the error policy written down, or left to NODE_ENV?
3. Does every field that fans out use a per-request loader?
4. Do monitors and clients depend on the HTTP status of a GraphQL error?
5. Is introspection on or off on purpose?

Everything else

 Site https://code-with-sam-dev.github.io	 GitHub https://github.com/code-with-sam-dev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me sadza https://buymesadza.com/codewithsam
 X https://x.com/CodeWithSamDev	The Spring GraphQL deep dive https://youtu.be/7VgEcSodl6Q
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-graphql.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS GraphQL https://docs.nestjs.com/graphql/quick-start
 Facebook https://www.facebook.com/codewithsamdev	

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. GraphQL is a trademark of the GraphQL Foundation. Apollo is a trademark of Apollo Graph Inc. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.